

# Supabase SQL Reference

|          |                                        |
|----------|----------------------------------------|
| Project  | AVIORA Examination Portal              |
| Builder  | ZYXEN                                  |
| Version  | 1.0                                    |
| Date     | 23 July 2026                           |
| Database | Supabase PostgreSQL (Pro Plan)         |
| Tables   | 15 tables · 40+ indexes · 15 functions |

## ✗ CRITICAL

CONFIDENTIAL — This document contains the complete database schema and security configuration for the AVIORA examination portal. Do not share outside the ZYXEN development team. Run all SQL statements in Supabase SQL Editor IN THE ORDER SHOWN.

## ■ WARNING

ANTIGRAVITY BUG FIXES INCLUDED: 4 functions from Antigravity contained bugs that would cause production failures. All have been corrected in this document. Key fixes: (1) submit\_exam\_session — max\_score now uses SUM(eq.marks) instead of COUNT(\*)\*marks, GREATEST(0,...) floor applied to total\_score, time\_taken uses session wall-clock not answer sum. (2) create\_exam\_session — JSONB loop syntax corrected. (3) upsert\_question — column 'text' renamed to 'content', display\_order and created\_by added. (4) publish\_exam — audit log added, status auto-determined from type+schedule.

## Table of Contents

|    |                                        |
|----|----------------------------------------|
| 01 | EXTENSIONS & PREREQUISITES             |
| 02 | IMMUTABILITY TRIGGERS                  |
| 03 | UPDATED_AT AUTO-TRIGGER                |
| 04 | CORE TABLES                            |
| 05 | QUESTION BANK TABLES                   |
| 06 | EXAM TABLES                            |
| 07 | SESSION & ANSWER TABLES                |
| 08 | RESULT & SECURITY TABLES               |
| 09 | AUDIT & SESSION CONTROL TABLES         |
| 10 | COMPLETE INDEX DEFINITIONS             |
| 11 | ROW LEVEL SECURITY (RLS)               |
| 12 | DATABASE FUNCTIONS (STORED PROCEDURES) |
| 13 | FUNCTION GRANTS & RPC ACCESS           |
| 14 | STORAGE BUCKET SETUP                   |
| 15 | VERIFICATION QUERIES                   |

## SECTION 1 EXTENSIONS & PREREQUISITES

### ■ INFO

Run these first before any table creation. Extensions must be enabled at the database level.

```
-- Enable UUID generation (required for gen_random_uuid())  
CREATE EXTENSION IF NOT EXISTS "uuid-oss";  
  
-- Enable trigram search (required for full-text student name search)  
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
```

## SECTION 2 IMMUTABILITY TRIGGERS

### IMMUTABILITY TRIGGERS

```
CREATE OR REPLACE FUNCTION reject_mutation()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
SECURITY DEFINER  
SET search_path = public, pg_catalog  
AS $$  
BEGIN  
    RAISE EXCEPTION 'IMMUTABLE_TABLE: % cannot be modified after insert. Table: %, Operation: %',  
        TG_TABLE_NAME, TG_TABLE_NAME, TG_OP;  
END;  
$$;
```

## SECTION 3 UPDATED\_AT AUTO-TRIGGER

### ■ INFO

This single function is reused by ALL tables that have an `updated_at` column. Create it before any tables.

```
-- Universal updated_at trigger function
-- Applied to every table that has an updated_at column.
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_catalog
AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$;
```

## SECTION 4 CORE TABLES

### TABLE: users

Universal identity table. Every person in the system (student, admin, super\_admin) is a row here. Linked to Supabase Auth via matching UUID.

```
CREATE TABLE IF NOT EXISTS users (
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  email       text          UNIQUE NOT NULL,
  role        text          NOT NULL
              CHECK (role IN ('student', 'admin', 'super_admin')),
  status      text          NOT NULL DEFAULT 'active'
              CHECK (status IN ('active', 'suspended', 'deactivated')),
  force_password_change boolean NOT NULL DEFAULT true,
  created_at  timestampz    NOT NULL DEFAULT now(),
  updated_at  timestampz    NOT NULL DEFAULT now(),
  deleted_at  timestampz    NULL
);

CREATE TRIGGER users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

### TABLE: batches

Named groups of students. Used for bulk exam enrollment and reporting. Maps to course cohorts in a future LMS.

```
CREATE TABLE IF NOT EXISTS batches (
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text          NOT NULL,
  description text          NULL,
  status      text          NOT NULL DEFAULT 'active'
              CHECK (status IN ('active', 'archived')),
  created_at  timestampz    NOT NULL DEFAULT now(),
  updated_at  timestampz    NOT NULL DEFAULT now(),
  deleted_at  timestampz    NULL
);

CREATE TRIGGER batches_updated_at
BEFORE UPDATE ON batches
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

### TABLE: student\_profiles

Extended profile for students only. Separated from users to keep the identity table generic and LMS-ready.

```
CREATE TABLE IF NOT EXISTS student_profiles (
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid          NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,
  full_name   text          NOT NULL,
  roll_number text          NOT NULL UNIQUE,
  batch_id    uuid          NULL REFERENCES batches(id) ON DELETE SET NULL,
  photo_url   text          NULL,
  phone       text          NULL,
  metadata    jsonb         NOT NULL DEFAULT '{}',
  created_at  timestampz    NOT NULL DEFAULT now(),
  updated_at  timestampz    NOT NULL DEFAULT now(),
  deleted_at  timestampz    NULL
);

CREATE TRIGGER student_profiles_updated_at
BEFORE UPDATE ON student_profiles
```

```
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

## SECTION 5 QUESTION BANK TABLES

**TABLE: question\_banks**

Container for questions, organised by subject. One bank per subject. Reusable across multiple exams.

```
CREATE TABLE IF NOT EXISTS question_banks (
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text          NOT NULL,
  subject     text          NOT NULL,
  description  text          NULL,
  status      text          NOT NULL DEFAULT 'active'
    CHECK (status IN ('active', 'archived')),
  created_by  uuid          NOT NULL REFERENCES users(id),
  created_at  timestampz    NOT NULL DEFAULT now(),
  updated_at  timestampz    NOT NULL DEFAULT now(),
  deleted_at  timestampz    NULL
);

CREATE TRIGGER question_banks_updated_at
BEFORE UPDATE ON question_banks
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

**TABLE: questions**

Individual MCQ questions. Minimum 10-char content. Explanation (min 20 chars) is mandatory — shown to students on review screen.

```
CREATE TABLE IF NOT EXISTS questions (
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  bank_id     uuid          NOT NULL REFERENCES question_banks(id) ON DELETE RESTRICT,
  content     text          NOT NULL CHECK (length(content) >= 10),
  type        text          NOT NULL DEFAULT 'mcq' CHECK (type IN ('mcq')),
  difficulty  text          NOT NULL DEFAULT 'medium'
    CHECK (difficulty IN ('easy', 'medium', 'hard')),
  subject     text          NOT NULL,
  topic       text          NULL,
  tags        text[]        NOT NULL DEFAULT '{}',
  explanation  text          NULL CHECK (explanation IS NULL OR length(explanation) >= 20),
  metadata    jsonb         NOT NULL DEFAULT '{}',
  created_by  uuid          NOT NULL REFERENCES users(id),
  updated_by  uuid          NOT NULL REFERENCES users(id),
  created_at  timestampz    NOT NULL DEFAULT now(),
  updated_at  timestampz    NOT NULL DEFAULT now(),
  deleted_at  timestampz    NULL
);

CREATE TRIGGER questions_updated_at
BEFORE UPDATE ON questions
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

**TABLE: question\_options**

### ■ WARNING

CRITICAL: The correct answer is ALWAYS identified by option UUID, NEVER by position (A/B/C/D). The unique partial index below enforces exactly one correct option per question at the database level.



```
CREATE TABLE IF NOT EXISTS question_options (  
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),  
  question_id uuid          NOT NULL REFERENCES questions(id) ON DELETE CASCADE,  
  content     text          NOT NULL CHECK (length(content) >= 1),  
  is_correct  boolean       NOT NULL DEFAULT false,  
  display_order smallint    NOT NULL DEFAULT 0,  
  created_at  timestampz    NOT NULL DEFAULT now(),  
  updated_at  timestampz    NOT NULL DEFAULT now()  
);  
  
CREATE TRIGGER question_options_updated_at  
  BEFORE UPDATE ON question_options  
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();  
  
-- DATABASE-LEVEL CONSTRAINT: exactly ONE correct option per question  
CREATE UNIQUE INDEX IF NOT EXISTS idx_question_options_one_correct  
  ON question_options(question_id)  
  WHERE is_correct = true;
```

## SECTION 6 EXAM TABLES

### TABLE: exams

Each exam record. Covers both practice (available anytime) and scheduled (time-windowed) types. Settings stored as JSONB for zero-migration extensibility.

```
CREATE TABLE IF NOT EXISTS exams (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  bank_id           uuid          NOT NULL REFERENCES question_banks(id) ON DELETE RESTRICT,
  title             text          NOT NULL,
  subject           text          NOT NULL,
  description        text          NULL,
  instructions       text          NULL,
  type              text          NOT NULL CHECK (type IN ('practice', 'scheduled')),
  duration_minutes  integer       NOT NULL CHECK (duration_minutes > 0),
  total_questions   integer       NOT NULL CHECK (total_questions > 0),
  marks_per_question numeric(5,2) NOT NULL DEFAULT 1.0 CHECK (marks_per_question > 0),
  negative_marks     numeric(5,2) NOT NULL DEFAULT 0.0 CHECK (negative_marks >= 0),
  passing_marks     numeric(7,2)  NULL CHECK (passing_marks >= 0),
  status            text          NOT NULL DEFAULT 'draft'
                    CHECK (status IN ('draft', 'scheduled', 'active', 'completed', 'archived')),
  scheduled_at      timestamptz  NULL,
  ends_at           timestamptz  NULL,
  settings           jsonb        NOT NULL DEFAULT '{
    "randomize_questions": true,
    "randomize_options": true,
    "fullscreen_required": true,
    "max_tab_switches": 5,
    "auto_submit_on_max_violations": false,
    "show_result_immediately": true,
    "allow_question_review": true,
    "show_leaderboard_after": "exam_end",
    "watermark_enabled": true
  }::jsonb',
  created_by        uuid          NOT NULL REFERENCES users(id),
  updated_by        uuid          NOT NULL REFERENCES users(id),
  created_at        timestamptz  NOT NULL DEFAULT now(),
  updated_at        timestamptz  NOT NULL DEFAULT now(),
  deleted_at        timestamptz  NULL,
  CONSTRAINT scheduled_exam_requires_times CHECK (
    type != 'scheduled' OR (scheduled_at IS NOT NULL AND ends_at IS NOT NULL)
  ),
  CONSTRAINT ends_must_be_after_start CHECK (
    ends_at IS NULL OR scheduled_at IS NULL OR ends_at > scheduled_at
  )
);

CREATE TRIGGER exams_updated_at
BEFORE UPDATE ON exams
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

### TABLE: exam\_questions

IMMUTABLE snapshot of questions selected for an exam. Created once when exam is published. Never modified. This means editing a source question does not affect live or completed exams.

```
CREATE TABLE IF NOT EXISTS exam_questions (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  exam_id           uuid          NOT NULL REFERENCES exams(id) ON DELETE RESTRICT,
  question_id       uuid          NOT NULL REFERENCES questions(id) ON DELETE RESTRICT,
  base_order        smallint      NOT NULL,
  marks             numeric(5,2)  NOT NULL DEFAULT 1.0,
  created_at        timestamptz  NOT NULL DEFAULT now()
  -- No updated_at — this table is immutable after insert
);
```

```
CREATE TRIGGER eq_immutable BEFORE UPDATE OR DELETE ON exam_questions FOR EACH ROW EXECUTE FUNCTION reject_mutation
```

## TABLE: exam\_enrollments

Controls which students can access which exams. A student cannot start an exam without an enrollment row.

```
CREATE TABLE IF NOT EXISTS exam_enrollments (  
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),  
  exam_id     uuid          NOT NULL REFERENCES exams(id) ON DELETE CASCADE,  
  student_id  uuid          NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  enrolled_by uuid          NOT NULL REFERENCES users(id),  
  created_at  timestampz    NOT NULL DEFAULT now()  
);
```

## SECTION 7 SESSION & ANSWER TABLES

**TABLE: exam\_sessions**

### ■ WARNING

THE MOST CRITICAL TABLE. One row per student per exam attempt. question\_order and option\_orders store the randomisation mapping — generated ONCE at session start, NEVER regenerated. The unique partial index enforces one active/submitted session per student per exam.

```
CREATE TABLE IF NOT EXISTS exam_sessions (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  exam_id           uuid          NOT NULL REFERENCES exams(id) ON DELETE RESTRICT,
  student_id        uuid          NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  status            text          NOT NULL DEFAULT 'active'
                                CHECK (status IN ('active', 'submitted', 'expired', 'terminated')),
  started_at        timestampz    NOT NULL DEFAULT now(),
  expires_at        timestampz    NOT NULL,
  submitted_at      timestampz    NULL,
  last_synced_at    timestampz    NULL,
  -- Stores shuffled question UUID array for THIS student: ["uuid-q5", "uuid-q2", ...]
  question_order    jsonb         NOT NULL DEFAULT '[]' CHECK (jsonb_typeof(question_order) = 'array'),
  -- Stores per-question shuffled option arrays: {"uuid-q5": ["uuid-opt-b", "uuid-opt-a", ...]}
  option_orders     jsonb         NOT NULL DEFAULT '{}' CHECK (jsonb_typeof(option_orders) = 'object'),
  submission_token   uuid          NOT NULL DEFAULT gen_random_uuid(),
  device_info        jsonb         NOT NULL DEFAULT '{}',
  ip_address         inet         NULL,
  security_violations integer      NOT NULL DEFAULT 0 CHECK (security_violations >= 0),
  created_at         timestampz    NOT NULL DEFAULT now(),
  updated_at         timestampz    NOT NULL DEFAULT now()
);

CREATE TRIGGER exam_sessions_updated_at
BEFORE UPDATE ON exam_sessions
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- ONE active OR submitted session per student per exam (database enforced)
CREATE UNIQUE INDEX IF NOT EXISTS idx_exam_sessions_one_active
ON exam_sessions(exam_id, student_id)
WHERE status IN ('active', 'submitted');
```

**TABLE: student\_answers**

One row per student per question per session. Upserted on every autosave batch. The sync\_status column lives ONLY in IndexedDB on the client — it does not exist here.

```
CREATE TABLE IF NOT EXISTS student_answers (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id        uuid          NOT NULL REFERENCES exam_sessions(id) ON DELETE CASCADE,
  question_id        uuid          NOT NULL REFERENCES questions(id) ON DELETE RESTRICT,
  selected_option_id uuid          NULL REFERENCES question_options(id) ON DELETE RESTRICT,
  is_marked_for_review boolean      NOT NULL DEFAULT false,
  is_visited         boolean      NOT NULL DEFAULT false,
  time_spent_seconds integer        NOT NULL DEFAULT 0 CHECK (time_spent_seconds >= 0),
  answered_at        timestampz    NULL,
  updated_at         timestampz    NOT NULL DEFAULT now()
);

CREATE TRIGGER student_answers_updated_at
BEFORE UPDATE ON student_answers
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```



## SECTION 8 RESULT & SECURITY TABLES

**TABLE: exam\_results**

Computed ONCE on submission by the submit\_exam\_session() function. Never recomputed on page load. result\_data JSONB contains the full per-question breakdown including explanations.

```
CREATE TABLE IF NOT EXISTS exam_results (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id        uuid          NOT NULL UNIQUE REFERENCES exam_sessions(id) ON DELETE RESTRICT,
  exam_id           uuid          NOT NULL REFERENCES exams(id) ON DELETE RESTRICT,
  student_id        uuid          NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  total_score        numeric(7,2) NOT NULL CHECK (total_score >= 0),
  max_score          numeric(7,2) NOT NULL CHECK (max_score >= 0),
  percentage         numeric(5,2) NOT NULL CHECK (percentage >= 0 AND percentage <= 100),
  correct_count      integer       NOT NULL CHECK (correct_count >= 0),
  incorrect_count    integer       NOT NULL CHECK (incorrect_count >= 0),
  unanswered_count   integer       NOT NULL CHECK (unanswered_count >= 0),
  time_taken_seconds integer       NOT NULL CHECK (time_taken_seconds >= 0),
  is_passed          boolean       NULL,
  computed_at        timestamptz  NOT NULL DEFAULT now(),
  result_data        jsonb         NOT NULL DEFAULT '{}',
  -- result_data structure:
  -- { "questions": [{ "question_id", "question_content", "selected_option_id",
  -- "correct_option_id", "is_correct", "is_unanswered", "marks_awarded",
  -- "explanation", "time_spent_seconds" }, ...] }
);

CREATE TRIGGER er_immutable BEFORE UPDATE OR DELETE ON exam_results FOR EACH ROW EXECUTE FUNCTION reject_mutation
```

**TABLE: security\_events**

All anti-cheating events logged during an exam. Synced from client IndexedDB via the autosave batch. Admins view these in the monitoring dashboard.

```
CREATE TABLE IF NOT EXISTS security_events (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id        uuid          NOT NULL REFERENCES exam_sessions(id) ON DELETE CASCADE,
  student_id        uuid          NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  event_type         text          NOT NULL CHECK (event_type IN (
    'fullscreen_exit', 'tab_switch', 'focus_loss',
    'right_click_attempt', 'keyboard_shortcut_blocked',
    'copy_attempt', 'paste_attempt', 'context_menu_blocked'
  )),
  duration_seconds   integer       NULL CHECK (duration_seconds >= 0),
  event_data         jsonb         NOT NULL DEFAULT '{}',
  occurred_at        timestamptz  NOT NULL DEFAULT now()
);
```

## SECTION 9 AUDIT & SESSION CONTROL TABLES

### TABLE: active\_sessions

Single-device enforcement. One active row per user at any time. On new login, all existing rows for that user are terminated. Middleware checks this table on every authenticated request.

```
CREATE TABLE IF NOT EXISTS active_sessions (
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid          NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  text          NOT NULL,
  device_info jsonb         NOT NULL DEFAULT '{}',
  ip_address  inet          NULL,
  status      text          NOT NULL DEFAULT 'active'
    CHECK (status IN ('active', 'terminated')),
  created_at  timestampz    NOT NULL DEFAULT now(),
  last_active_at timestampz NOT NULL DEFAULT now(),
  expires_at  timestampz    NOT NULL,
  updated_at  timestampz    NOT NULL DEFAULT now()
);

CREATE TRIGGER active_sessions_updated_at
BEFORE UPDATE ON active_sessions
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

### TABLE: audit\_logs

Immutable append-only log of every meaningful action. Only the service role key can INSERT here. Never modified after insert. The foundation of your audit trail.

```
CREATE TABLE IF NOT EXISTS audit_logs (
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id    uuid          NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  actor_role  text          NOT NULL,
  action      text          NOT NULL,
  resource_type text        NOT NULL,
  resource_id uuid          NULL,
  metadata    jsonb         NOT NULL DEFAULT '{}',
  ip_address  inet          NULL,
  created_at  timestampz    NOT NULL DEFAULT now()
  -- NO updated_at — audit logs are immutable
);

CREATE TRIGGER al_immutable BEFORE UPDATE OR DELETE ON audit_logs FOR EACH ROW EXECUTE FUNCTION reject_mutation
```

## SECTION 10 COMPLETE INDEX DEFINITIONS

### ■ INFO

All indexes use IF NOT EXISTS so this section is safe to re-run. Partial indexes (WHERE clause) dramatically reduce index size by excluding soft-deleted rows and completed sessions.

### users & student\_profiles

```
CREATE INDEX IF NOT EXISTS idx_users_email
  ON users(email) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_users_role
  ON users(role) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_users_status
  ON users(status) WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_student_profiles_user_id
  ON student_profiles(user_id);
CREATE INDEX IF NOT EXISTS idx_student_profiles_roll_number
  ON student_profiles(roll_number) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_student_profiles_batch_id
  ON student_profiles(batch_id) WHERE deleted_at IS NULL;
-- Trigram index for fast name search (requires pg_trgm extension)
CREATE INDEX IF NOT EXISTS idx_student_profiles_name_trgm
  ON student_profiles USING gin(full_name gin_trgm_ops)
  WHERE deleted_at IS NULL;
```

### batches, question\_banks, questions, question\_options

```
CREATE INDEX IF NOT EXISTS idx_batches_status
  ON batches(status) WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_question_banks_subject
  ON question_banks(subject) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_question_banks_status
  ON question_banks(status) WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_questions_bank_id
  ON questions(bank_id) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_questions_subject
  ON questions(subject) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_questions_difficulty
  ON questions(difficulty) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_questions_tags
  ON questions USING GIN(tags);

CREATE INDEX IF NOT EXISTS idx_question_options_question_id
  ON question_options(question_id);
```

### exams, exam\_questions, exam\_enrollments

```
CREATE INDEX IF NOT EXISTS idx_exams_type
  ON exams(type) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_exams_status
  ON exams(status) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_exams_subject
  ON exams(subject) WHERE deleted_at IS NULL;
CREATE INDEX IF NOT EXISTS idx_exams_bank_id
  ON exams(bank_id) WHERE deleted_at IS NULL;
```



```
-- Partial index for scheduled exam queries
CREATE INDEX IF NOT EXISTS idx_exams_scheduled_at
  ON exams(scheduled_at)
  WHERE deleted_at IS NULL AND type = 'scheduled';

CREATE INDEX IF NOT EXISTS idx_exam_questions_exam_id
  ON exam_questions(exam_id);
CREATE UNIQUE INDEX IF NOT EXISTS idx_exam_questions_unique
  ON exam_questions(exam_id, question_id);

CREATE INDEX IF NOT EXISTS idx_exam_enrollments_exam_id
  ON exam_enrollments(exam_id);
CREATE INDEX IF NOT EXISTS idx_exam_enrollments_student_id
  ON exam_enrollments(student_id);
CREATE UNIQUE INDEX IF NOT EXISTS idx_exam_enrollments_unique
  ON exam_enrollments(exam_id, student_id);
```

## exam\_sessions, student\_answers

```
CREATE INDEX IF NOT EXISTS idx_exam_sessions_student_id
  ON exam_sessions(student_id);
CREATE INDEX IF NOT EXISTS idx_exam_sessions_student_status
  ON exam_sessions(student_id, status);
CREATE INDEX IF NOT EXISTS idx_exam_sessions_exam_id
  ON exam_sessions(exam_id);
CREATE INDEX IF NOT EXISTS idx_exam_sessions_status
  ON exam_sessions(status);
-- For cron job: finding expired active sessions
CREATE INDEX IF NOT EXISTS idx_exam_sessions_expires_at
  ON exam_sessions(expires_at) WHERE status = 'active';
-- Composite for ownership checks (student_id + exam_id)
CREATE INDEX IF NOT EXISTS idx_exam_sessions_exam_student
  ON exam_sessions(exam_id, student_id);

CREATE INDEX IF NOT EXISTS idx_student_answers_session_id
  ON student_answers(session_id);
CREATE INDEX IF NOT EXISTS idx_student_answers_question_id
  ON student_answers(question_id);
-- Unique answer per question per session
CREATE UNIQUE INDEX IF NOT EXISTS idx_student_answers_unique
  ON student_answers(session_id, question_id);
```

## exam\_results, security\_events, active\_sessions, audit\_logs

```
CREATE INDEX IF NOT EXISTS idx_exam_results_student_id
  ON exam_results(student_id);
CREATE INDEX IF NOT EXISTS idx_exam_results_exam_id
  ON exam_results(exam_id);
-- For leaderboard queries – sorted by score descending
CREATE INDEX IF NOT EXISTS idx_exam_results_exam_percentage
  ON exam_results(exam_id, percentage DESC);

CREATE INDEX IF NOT EXISTS idx_security_events_session_id
  ON security_events(session_id);
CREATE INDEX IF NOT EXISTS idx_security_events_student_id
  ON security_events(student_id);
CREATE INDEX IF NOT EXISTS idx_security_events_occurred_at
  ON security_events(occurred_at);

CREATE INDEX IF NOT EXISTS idx_active_sessions_user_id
  ON active_sessions(user_id) WHERE status = 'active';
CREATE INDEX IF NOT EXISTS idx_active_sessions_token_hash
  ON active_sessions(token_hash) WHERE status = 'active';
CREATE INDEX IF NOT EXISTS idx_active_sessions_expires_at
  ON active_sessions(expires_at) WHERE status = 'active';
CREATE INDEX IF NOT EXISTS idx_active_sessions_user_status
  ON active_sessions(user_id, status);

CREATE INDEX IF NOT EXISTS idx_audit_logs_actor_id
```

```
ON audit_logs(actor_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_resource
ON audit_logs(resource_type, resource_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_action
ON audit_logs(action);
CREATE INDEX IF NOT EXISTS idx_audit_logs_created_at
ON audit_logs(created_at DESC);
CREATE INDEX IF NOT EXISTS idx_audit_logs_actor_time
ON audit_logs(actor_id, created_at DESC);
```

## SECTION 11 ROW LEVEL SECURITY (RLS)

### ✗ CRITICAL

Run `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` on EVERY table. Default policy after enabling RLS is `DENY ALL`. Explicit policies then grant specific access. Missing a table means no data protection.

### Enable RLS on all tables

```
ALTER TABLE users          ENABLE ROW LEVEL SECURITY;
ALTER TABLE student_profiles  ENABLE ROW LEVEL SECURITY;
ALTER TABLE batches         ENABLE ROW LEVEL SECURITY;
ALTER TABLE question_banks   ENABLE ROW LEVEL SECURITY;
ALTER TABLE questions        ENABLE ROW LEVEL SECURITY;
ALTER TABLE question_options ENABLE ROW LEVEL SECURITY;
ALTER TABLE exams            ENABLE ROW LEVEL SECURITY;
ALTER TABLE exam_questions   ENABLE ROW LEVEL SECURITY;
ALTER TABLE exam_enrollments ENABLE ROW LEVEL SECURITY;
ALTER TABLE exam_sessions    ENABLE ROW LEVEL SECURITY;
ALTER TABLE student_answers  ENABLE ROW LEVEL SECURITY;
ALTER TABLE exam_results     ENABLE ROW LEVEL SECURITY;
ALTER TABLE security_events  ENABLE ROW LEVEL SECURITY;
ALTER TABLE active_sessions  ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_logs       ENABLE ROW LEVEL SECURITY;
```

### Admin helper function

SECURITY DEFINER function that checks if the calling user is an active admin. Used in every admin policy to avoid repetition and ensure consistency.

```
CREATE OR REPLACE FUNCTION is_admin()
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
STABLE
SET search_path = public, pg_catalog
AS $$
BEGIN
    RETURN EXISTS (
        SELECT 1 FROM users
        WHERE id = auth.uid()
        AND role IN ('admin', 'super_admin')
        AND status = 'active'
        AND deleted_at IS NULL
    );
END;
$$;
```

### Policies: users

```
-- Students read only their own row
CREATE POLICY "users_student_read_own" ON users
FOR SELECT TO authenticated
USING (id = auth.uid());

-- Admins read all non-deleted users
CREATE POLICY "users_admin_read_all" ON users
FOR SELECT TO authenticated
```

```

    USING (is_admin() AND deleted_at IS NULL);

-- Only admins can create users (student accounts)
CREATE POLICY "users_admin_insert" ON users
  FOR INSERT TO authenticated
  WITH CHECK (is_admin());

-- Admins can update any user record
CREATE POLICY "users_admin_update" ON users
  FOR UPDATE TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- Direct student update policy removed for security.
-- Use update_force_password_change RPC instead.

```

## Policies: student\_profiles, batches, question\_banks, questions, question\_options

```

-- student_profiles: students read own, admins full access
CREATE POLICY "sp_student_read_own" ON student_profiles
  FOR SELECT TO authenticated USING (user_id = auth.uid());
CREATE POLICY "sp_admin_all" ON student_profiles
  FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- batches: students read active ones, admins full access
CREATE POLICY "batches_student_read" ON batches
  FOR SELECT TO authenticated
  USING (status = 'active' AND deleted_at IS NULL);
CREATE POLICY "batches_admin_all" ON batches
  FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- question_banks: STUDENTS CANNOT ACCESS – admins only
CREATE POLICY "qb_admin_all" ON question_banks
  FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- questions: STUDENTS CANNOT READ DIRECTLY – admins only
-- Students receive question content only through exam session API response
CREATE POLICY "questions_admin_all" ON questions
  FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- question_options: STUDENTS CANNOT READ – is_correct field must NEVER reach client
CREATE POLICY "question_options_admin_all" ON question_options
  FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

```

## Policies: exams, exam\_questions, exam\_enrollments

```

-- exams: students read active/completed only; admins full access
CREATE POLICY "exams_student_read" ON exams
  FOR SELECT TO authenticated
  USING (
    status IN ('active', 'completed')
    AND deleted_at IS NULL
    AND EXISTS (SELECT 1 FROM users u WHERE u.id = auth.uid() AND u.role = 'student')
  );
CREATE POLICY "exams_admin_all" ON exams
  FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- exam_questions: students CANNOT read (snapshot is server-only)
CREATE POLICY "exam_questions_admin_all" ON exam_questions
  FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- exam_enrollments: students read their own; admins full access
CREATE POLICY "ee_student_read_own" ON exam_enrollments

```

```

FOR SELECT TO authenticated USING (student_id = auth.uid());
CREATE POLICY "ee_admin_all" ON exam_enrollments
FOR ALL TO authenticated
USING (is_admin()) WITH CHECK (is_admin());

```

## Policies: exam\_sessions, student\_answers

```

-- exam_sessions: students read/update their own active sessions
CREATE POLICY "es_student_read_own" ON exam_sessions
FOR SELECT TO authenticated USING (student_id = auth.uid());
-- es_student_update_own removed; all session state changes go through RPCs.
CREATE POLICY "es_admin_all" ON exam_sessions
FOR ALL TO authenticated
USING (is_admin()) WITH CHECK (is_admin());

-- student_answers: students read/write own (only if session is active)
CREATE POLICY "sa_student_read_own" ON student_answers
FOR SELECT TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM exam_sessions es
    WHERE es.id = session_id AND es.student_id = auth.uid()
  )
);
CREATE POLICY "sa_student_insert_own_active" ON student_answers
FOR INSERT TO authenticated
WITH CHECK (
  EXISTS (
    SELECT 1 FROM exam_sessions es
    WHERE es.id = session_id
    AND es.student_id = auth.uid()
    AND es.status = 'active'
  )
);
CREATE POLICY "sa_student_update_own_active" ON student_answers
FOR UPDATE TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM exam_sessions es
    WHERE es.id = session_id
    AND es.student_id = auth.uid()
    AND es.status = 'active'
  )
)
WITH CHECK (
  EXISTS (
    SELECT 1 FROM exam_sessions es
    WHERE es.id = session_id
    AND es.student_id = auth.uid()
    AND es.status = 'active'
  )
);
CREATE POLICY "sa_admin_read_all" ON student_answers
FOR SELECT TO authenticated USING (is_admin());

```

## Policies: exam\_results, security\_events, active\_sessions, audit\_logs

```

-- exam_results: students read their own; admins read all; only service role inserts
CREATE POLICY "er_student_read_own" ON exam_results
FOR SELECT TO authenticated USING (student_id = auth.uid());
CREATE POLICY "er_admin_read_all" ON exam_results
FOR SELECT TO authenticated USING (is_admin());

-- security_events: admins read only; inserts via service role in API routes
CREATE POLICY "se_admin_read_all" ON security_events
FOR SELECT TO authenticated USING (is_admin());

-- active_sessions: users read their own active session

```

```
CREATE POLICY "as_user_read_own" ON active_sessions
  FOR SELECT TO authenticated
  USING (user_id = auth.uid() AND status = 'active');

-- audit_logs: only admins can read; only service role can insert
CREATE POLICY "al_admin_read" ON audit_logs
  FOR SELECT TO authenticated USING (is_admin());
```

## SECTION 12

## DATABASE FUNCTIONS (STORED PROCEDURES)

## ■ INFO

All functions use SECURITY DEFINER — they run as the function owner (bypassing RLS). Security logic is enforced INSIDE each function. All are called via Supabase RPC from Next.js API routes using the service role key.

## ■ WARNING

ANTIGRAVITY BUG FIXES APPLIED: (1) create\_exam\_session — fixed JSONB loop syntax. (2) submit\_exam\_session — fixed max\_score to use SUM(eq.marks), added GREATEST(0,...) floor, fixed time\_taken from session timestamps, added expired status handling. (3) upsert\_question — fixed column name 'text' → 'content', added display\_order, added created\_by/updated\_by. (4) publish\_exam — added audit log, added smart status determination.

**FUNCTION: update\_force\_password\_change**

Allows an authenticated user to clear their force\_password\_change flag. Strictly isolated to prevent modifications to other columns or other users.

```
CREATE OR REPLACE FUNCTION update_force_password_change(p_user_id uuid)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_catalog
AS $$
BEGIN
    -- Only allow authenticated user to update their own row
    IF auth.uid() != p_user_id THEN
        RAISE EXCEPTION 'UNAUTHORIZED';
    END IF;

    UPDATE users
    SET force_password_change = false, updated_at = now()
    WHERE id = p_user_id;

    -- Optional: Audit logging if it aligns with the rest of the architecture
    INSERT INTO audit_logs (actor_id, actor_role, action, resource_type, resource_id)
    VALUES (
        p_user_id, 'student', 'user.password_changed', 'user', p_user_id
    );
END;
$;
```

**FUNCTION: create\_exam\_session**

Called when a student clicks Start Exam. Atomically creates the session + inserts one student\_answers row per question + writes audit log.

```
CREATE OR REPLACE FUNCTION create_exam_session(
    p_exam_id      uuid,
    p_student_id    uuid,
    p_duration_minutes integer,
    p_question_order jsonb, -- shuffled array of question UUIDs
    p_option_orders  jsonb, -- { question_id: [option_id, ...] }
    p_device_info    jsonb,
```

```

    p_ip_address      text,
    p_student_role    text
) RETURNS jsonb
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE
    v_session_id      uuid;
    v_submission_token uuid      := gen_random_uuid();
    v_expires_at       timestampz := now() + (p_duration_minutes * interval '1 minute');

BEGIN
    -- 1. question_order is a non-empty JSON array
    IF jsonb_typeof(p_question_order) != 'array' OR jsonb_array_length(p_question_order) = 0 THEN
        RAISE EXCEPTION 'INVALID_QUESTION_ORDER';
    END IF;

    -- 2. Every element is a valid UUID & 3. No duplicates
    IF (
        SELECT COUNT(*) FROM jsonb_array_elements_text(p_question_order)
    ) != (
        SELECT COUNT(DISTINCT value) FROM jsonb_array_elements_text(p_question_order)
    ) THEN
        RAISE EXCEPTION 'INVALID_QUESTION_ORDER';
    END IF;

    -- Confirm all question IDs belong to this exam's snapshot
    IF EXISTS (
        SELECT 1
        FROM jsonb_array_elements_text(p_question_order) AS elem
        WHERE NOT EXISTS (
            SELECT 1 FROM exam_questions
            WHERE exam_id = p_exam_id
            AND question_id = elem::uuid
        )
    ) THEN
        RAISE EXCEPTION 'VALIDATION_QUESTION_NOT_IN_EXAM';
    END IF;

    -- 4. option_orders is a JSON object
    IF jsonb_typeof(p_option_orders) != 'object' THEN
        RAISE EXCEPTION 'INVALID_OPTION_ORDERS';
    END IF;

    -- 5. & 6. option_orders keys exactly match question_order elements
    IF (
        SELECT COUNT(*) FROM jsonb_array_elements_text(p_question_order) AS q
        WHERE NOT p_option_orders ? q
    ) > 0 OR (
        SELECT COUNT(*) FROM jsonb_object_keys(p_option_orders) AS k
        WHERE NOT p_question_order @> to_jsonb(k)
    ) > 0 THEN
        RAISE EXCEPTION 'INVALID_OPTION_ORDERS';
    END IF;

    -- 7. Every option UUID supplied in option_orders belongs to its associated question
    IF EXISTS (
        SELECT 1
        FROM jsonb_each(p_option_orders) AS qo(q_id, opts),
        jsonb_array_elements_text(opts) AS o(opt_id)
        WHERE NOT EXISTS (
            SELECT 1 FROM question_options
            WHERE id = opt_id::uuid AND question_id = qo.q_id::uuid
        )
    ) THEN
        RAISE EXCEPTION 'INVALID_OPTION_ORDERS';
    END IF;

    -- Insert the exam session with exception-based concurrency control
    BEGIN
        INSERT INTO exam_sessions (
            exam_id, student_id, status, started_at, expires_at,
            question_order, option_orders, submission_token, device_info, ip_address
        ) VALUES (
            p_exam_id, p_student_id, 'active', now(), v_expires_at,
            p_question_order, p_option_orders, v_submission_token,
            p_device_info, p_ip_address::inet
        ) RETURNING id INTO v_session_id;
    EXCEPTION WHEN unique_violation THEN

```



```

    RAISE EXCEPTION 'ALREADY_EXISTS';
END;

-- Insert one student_answers placeholder per question
INSERT INTO student_answers (
    session_id, question_id,
    selected_option_id, is_visited, is_marked_for_review, time_spent_seconds
)
SELECT v_session_id, (elem::text)::uuid, NULL, false, false, 0
FROM jsonb_array_elements_text(p_question_order) AS elem;

-- Audit log
INSERT INTO audit_logs (actor_id, actor_role, action, resource_type, resource_id, ip_address)
VALUES (
    p_student_id, p_student_role, 'exam.session_created',
    'exam', p_exam_id, p_ip_address::inet
);

RETURN jsonb_build_object(
    'id', v_session_id,
    'exam_id', p_exam_id,
    'started_at', now(),
    'expires_at', v_expires_at,
    'submission_token', v_submission_token,
    'status', 'active'
);
END;
$$;

```

## FUNCTION: compute\_and\_store\_result

Internal helper to compute results for an expiring or submitted session. Idempotent.

```

CREATE OR REPLACE FUNCTION compute_and_store_result(p_session_id uuid, p_student_id uuid)
RETURNS void
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE
    v_session      exam_sessions%ROWTYPE;
    v_exam         exams%ROWTYPE;
    v_total_score  numeric := 0;
    v_max_score    numeric := 0;
    v_correct_count integer := 0;
    v_incorrect_count integer := 0;
    v_unanswered_count integer := 0;
    v_time_taken   integer := 0;
    v_result_data  jsonb;
    v_is_passed    boolean;
    v_percentage   numeric;
BEGIN
    -- Idempotency check
    IF EXISTS (SELECT 1 FROM exam_results WHERE session_id = p_session_id) THEN
        RETURN;
    END IF;

    SELECT * INTO v_session FROM exam_sessions WHERE id = p_session_id;
    SELECT * INTO v_exam FROM exams WHERE id = v_session.exam_id;

    WITH answer_eval AS (
        SELECT
            sa.question_id,
            q.content,
            q.explanation,
            sa.selected_option_id,
            sel.content,
            cor.id,
            cor.content,
            sa.time_spent_seconds,
            eq.marks,
            (sa.selected_option_id IS NULL)
            (sel.id IS NOT NULL AND sel.is_correct = true)
            eq.base_order
        FROM student_answers sa
        INNER JOIN exam_questions eq
            AS question_content,
            AS selected_option_content,
            AS correct_option_id,
            AS correct_option_content,
            AS question_marks,
            AS is_unanswered,
            AS is_correct,
    )
    FROM student_answers sa
    INNER JOIN exam_questions eq

```

```

        ON eq.question_id = sa.question_id
        AND eq.exam_id    = v_session.exam_id
    INNER JOIN questions q ON q.id = sa.question_id
    LEFT JOIN question_options sel ON sel.id = sa.selected_option_id
    INNER JOIN question_options cor
        ON cor.question_id = sa.question_id AND cor.is_correct = true
    WHERE sa.session_id = p_session_id
),
scored AS (
    SELECT *,
        CASE
            WHEN is_unanswered THEN 0::numeric
            WHEN is_correct     THEN question_marks
            ELSE -(v_exam.negative_marks)
        END AS marks_awarded
    FROM answer_eval
)
SELECT
    GREATEST(0, COALESCE(SUM(marks_awarded), 0)),
    COALESCE(SUM(question_marks), 0),
    COUNT(*) FILTER (WHERE is_correct),
    COUNT(*) FILTER (WHERE NOT is_correct AND NOT is_unanswered),
    COUNT(*) FILTER (WHERE is_unanswered),
    jsonb_build_object(
        'questions',
        COALESCE(jsonb_agg(jsonb_build_object(
            'question_id',      question_id,
            'question_content', question_content,
            'selected_option_id', selected_option_id,
            'selected_option_content', selected_option_content,
            'correct_option_id', correct_option_id,
            'correct_option_content', correct_option_content,
            'is_correct',       is_correct,
            'is_unanswered',    is_unanswered,
            'marks_awarded',    marks_awarded,
            'explanation',       explanation,
            'time_spent_seconds', time_spent_seconds
        ) ORDER BY base_order ASC), '[]'::jsonb)
    )
    INTO
        v_total_score, v_max_score, v_correct_count,
        v_incorrect_count, v_unanswered_count, v_result_data
    FROM scored;

v_percentage := CASE
    WHEN v_max_score > 0 THEN ROUND((v_total_score / v_max_score) * 100, 2)
    ELSE 0
END;

v_is_passed := CASE
    WHEN v_exam.passing_marks IS NOT NULL THEN v_total_score >= v_exam.passing_marks
    ELSE NULL
END;

v_time_taken := LEAST(
    EXTRACT(EPOCH FROM (now() - v_session.started_at))::integer,
    (v_exam.duration_minutes * 60)
);

INSERT INTO exam_results (
    session_id, exam_id, student_id,
    total_score, max_score, percentage,
    correct_count, incorrect_count, unanswered_count,
    time_taken_seconds, is_passed, result_data
) VALUES (
    p_session_id, v_session.exam_id, p_student_id,
    v_total_score, v_max_score, v_percentage,
    v_correct_count, v_incorrect_count, v_unanswered_count,
    v_time_taken, v_is_passed, v_result_data
) ON CONFLICT DO NOTHING;
END;
$$;

```

## FUNCTION: submit\_exam\_session

Idempotent submission. Computes full result from `student_answers`, stores in `exam_results`, marks session submitted. If already submitted, returns existing result immediately.

```
CREATE OR REPLACE FUNCTION submit_exam_session(
  p_session_id      uuid,
  p_student_id      uuid,
  p_submission_token uuid,
  p_ip_address      text,
  p_student_role     text
) RETURNS jsonb
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE
  v_session      exam_sessions%ROWTYPE;
  v_exam         exams%ROWTYPE;
  v_result_id     uuid;
  v_total_score   numeric := 0;
  v_max_score     numeric := 0;
  v_correct_count integer := 0;
  v_incorrect_count integer := 0;
  v_unanswered_count integer := 0;
  v_time_taken    integer := 0;
  v_result_data   jsonb;
  v_is_passed     boolean;
  v_percentage    numeric;
  v_existing      jsonb;
BEGIN
  -- Fetch and validate session ownership with row lock
  SELECT * INTO v_session FROM exam_sessions
  WHERE id = p_session_id AND student_id = p_student_id
  FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'NOT_FOUND';
  END IF;

  -- Validate submission token
  IF v_session.submission_token != p_submission_token THEN
    RAISE EXCEPTION 'INVALID_TOKEN';
  END IF;

  -- Idempotency: return existing result if already submitted
  IF v_session.status = 'submitted' THEN
    SELECT jsonb_build_object(
      'id', id, 'session_id', session_id, 'exam_id', exam_id,
      'total_score', total_score, 'max_score', max_score,
      'percentage', percentage, 'is_passed', is_passed,
      'result_data', result_data
    ) INTO v_existing FROM exam_results WHERE session_id = p_session_id;
    RETURN v_existing;
  END IF;

  -- Accept 'active' or 'expired' (timer may have just run out)
  IF v_session.status NOT IN ('active', 'expired') THEN
    RAISE EXCEPTION 'SESSION_INVALID: status=%', v_session.status;
  END IF;

  -- Compute and store result
  PERFORM compute_and_store_result(p_session_id, p_student_id);

  -- Mark session submitted
  UPDATE exam_sessions
  SET status = 'submitted', submitted_at = now()
  WHERE id = p_session_id;

  SELECT id, total_score, max_score, percentage, is_passed, result_data
  INTO v_result_id, v_total_score, v_max_score, v_percentage, v_is_passed, v_result_data
  FROM exam_results WHERE session_id = p_session_id;

  -- Audit log
  INSERT INTO audit_logs (
    actor_id, actor_role, action, resource_type, resource_id, metadata, ip_address
  ) VALUES (
    p_student_id, p_student_role, 'exam.session_submitted',
    'exam_session', p_session_id,
    jsonb_build_object('score', v_total_score, 'percentage', v_percentage),
    p_ip_address::inet
  );
;
```

```

RETURN jsonb_build_object(
  'id',          v_result_id,
  'session_id',  p_session_id,
  'exam_id',     v_session.exam_id,
  'total_score', v_total_score,
  'max_score',   v_max_score,
  'percentage',  v_percentage,
  'is_passed',   v_is_passed,
  'result_data', v_result_data
);
END;
$$;

```

## FUNCTION: upsert\_question

Called from admin question editor. Handles create and update atomically. Enforces exactly 1 correct option, min 2 / max 6 options, manages display\_order.

```

CREATE OR REPLACE FUNCTION upsert_question(
  p_question_id uuid,      -- NULL to create new, UUID to update existing
  p_bank_id      uuid,
  p_content      text,
  p_subject      text,
  p_topic        text,
  p_difficulty   text,
  p_tags         text[],
  p_explanation    text,
  p_created_by   uuid,
  p_options      jsonb      -- [{id?: uuid, content: text, is_correct: boolean}]
) RETURNS uuid
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE
  v_question_id      uuid;
  v_correct_count     integer;
  v_option            jsonb;
  v_display_order     integer := 1;
BEGIN
  IF NOT is_admin() THEN
    RAISE EXCEPTION 'UNAUTHORIZED';
  END IF;

  -- Validate: exactly 1 correct option
  SELECT COUNT(*) INTO v_correct_count
  FROM jsonb_array_elements(p_options) AS opt
  WHERE (opt->>'is_correct')::boolean = true;
  IF v_correct_count != 1 THEN
    RAISE EXCEPTION 'VALIDATION: Exactly one option must be correct. Got: %', v_correct_count;
  END IF;

  -- Validate: option count
  IF jsonb_array_length(p_options) < 2 THEN
    RAISE EXCEPTION 'VALIDATION: Minimum 2 options required';
  END IF;
  IF jsonb_array_length(p_options) > 6 THEN
    RAISE EXCEPTION 'VALIDATION: Maximum 6 options allowed';
  END IF;

  -- Upsert question (uses 'content' column – not 'text')
  IF p_question_id IS NULL THEN
    INSERT INTO questions (
      bank_id, content, type, difficulty, subject, topic,
      tags, explanation, created_by, updated_by
    ) VALUES (
      p_bank_id, p_content, 'mcq', p_difficulty, p_subject, p_topic,
      p_tags, p_explanation, p_created_by, p_created_by
    ) RETURNING id INTO v_question_id;
  ELSE
    UPDATE questions SET
      content      = p_content,
      subject      = p_subject,
      topic        = p_topic,

```

```

        difficulty = p_difficulty,
        tags       = p_tags,
        explanation = p_explanation,
        updated_by  = p_created_by,
        updated_at  = now()
    WHERE id = p_question_id
    RETURNING id INTO v_question_id;
    IF v_question_id IS NULL THEN
        RAISE EXCEPTION 'NOT_FOUND: Question % not found', p_question_id;
    END IF;
END IF;
END IF;

-- Delete removed options (those not present in p_options by id)
DELETE FROM question_options
WHERE question_id = v_question_id
    AND id NOT IN (
        SELECT (opt->>'id')::uuid
        FROM jsonb_array_elements(p_options) AS opt
        WHERE opt->>'id' IS NOT NULL AND opt->>'id' != ''
    );

-- Upsert options with display_order
FOR v_option IN SELECT * FROM jsonb_array_elements(p_options) LOOP
    IF v_option->>'id' IS NULL OR v_option->>'id' = '' THEN
        -- New option
        INSERT INTO question_options (question_id, content, is_correct, display_order)
        VALUES (
            v_question_id,
            v_option->>'content',
            (v_option->>'is_correct')::boolean,
            v_display_order
        );
    ELSE
        -- Update existing option
        UPDATE question_options SET
            content      = v_option->>'content',
            is_correct    = (v_option->>'is_correct')::boolean,
            display_order = v_display_order,
            updated_at    = now()
        WHERE id = (v_option->>'id')::uuid AND question_id = v_question_id;
    END IF;
    v_display_order := v_display_order + 1;
END LOOP;

RETURN v_question_id;
END;
$$;

```

## FUNCTION: publish\_exam

Called from admin exam creation wizard Step 6. Atomically creates exam + snapshots questions into exam\_questions + creates enrollments. Idempotent enrollments (ON CONFLICT DO NOTHING).

```

CREATE OR REPLACE FUNCTION publish_exam(
    p_exam      jsonb,
    p_question_ids uuid[],
    p_student_ids uuid[],
    p_admin_id   uuid
) RETURNS uuid
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE
    v_exam_id   uuid;

    v_marks      numeric(5,2);
    v_exam_status text;
BEGIN
    IF NOT is_admin() THEN
        RAISE EXCEPTION 'UNAUTHORIZED';
    END IF;

    v_marks := COALESCE((p_exam->>'marks_per_question')::numeric, 1.0);

```

```

-- Determine status from type + schedule
IF p_exam->>'type' = 'practice' THEN
  v_exam_status := 'active';
ELSIF (p_exam->>'scheduled_at')::timestampz > now() THEN
  v_exam_status := 'scheduled';
ELSE
  v_exam_status := 'active';
END IF;

-- Create the exam
INSERT INTO exams (
  bank_id, title, subject, description, instructions, type,
  duration_minutes, total_questions, marks_per_question, negative_marks,
  passing_marks, status, scheduled_at, ends_at, settings, created_by, updated_by
) VALUES (
  (p_exam->>'bank_id')::uuid,
  p_exam->>'title', p_exam->>'subject',
  NULLIF(p_exam->>'description', ''),
  NULLIF(p_exam->>'instructions', ''),
  p_exam->>'type',
  (p_exam->>'duration_minutes')::integer,
  array_length(p_question_ids, 1),
  v_marks,
  COALESCE((p_exam->>'negative_marks')::numeric, 0.0),
  NULLIF(p_exam->>'passing_marks', '')::numeric,
  v_exam_status,
  NULLIF(p_exam->>'scheduled_at', '')::timestampz,
  NULLIF(p_exam->>'ends_at', '')::timestampz,
  COALESCE((p_exam->>'settings')::jsonb,
    '{"randomize_questions":true,"randomize_options":true,
      "fullscreen_required":true,"max_tab_switches":5,
      "auto_submit_on_max_violations":false,
      "show_result_immediately":true,"allow_question_review":true,
      "show_leaderboard_after":"exam_end","watermark_enabled":true}'::jsonb),
  p_admin_id, p_admin_id
) RETURNING id INTO v_exam_id;

-- Snapshot questions
INSERT INTO exam_questions (exam_id, question_id, base_order, marks)
SELECT v_exam_id, q_id, q_order, v_marks
FROM unnest(p_question_ids) WITH ORDINALITY AS t(q_id, q_order);

-- Create enrollments (idempotent)
INSERT INTO exam_enrollments (exam_id, student_id, enrolled_by)
SELECT v_exam_id, s_id, p_admin_id
FROM unnest(p_student_ids) AS s_id
ON CONFLICT (exam_id, student_id) DO NOTHING;

-- Audit log
INSERT INTO audit_logs (
  actor_id, actor_role, action, resource_type, resource_id, metadata
) VALUES (
  p_admin_id, 'admin', 'admin.exam_published', 'exam', v_exam_id,
  jsonb_build_object(
    'title', p_exam->>'title',
    'type', p_exam->>'type',
    'question_count', array_length(p_question_ids, 1),
    'student_count', array_length(p_student_ids, 1)
  )
);

RETURN v_exam_id;
END;
$$;

```

## FUNCTION: get\_monitoring\_data

Called every 30 seconds from admin monitoring page. Returns all enrolled students with their current session status for a given exam.

```

CREATE OR REPLACE FUNCTION get_monitoring_data(p_exam_id uuid)
RETURNS TABLE (

```

```

    full_name          text,
    roll_number        text,
    student_id         uuid,
    session_id         uuid,
    status             text,
    started_at         timestampz,
    expires_at         timestampz,
    submitted_at       timestampz,
    last_synced_at     timestampz,
    security_violations integer,
    enrolled_id        uuid
)
LANGUAGE plpgsql
SECURITY DEFINER
STABLE
SET search_path = public, pg_catalog
AS $$
BEGIN
    IF NOT is_admin() THEN
        RAISE EXCEPTION 'UNAUTHORIZED';
    END IF;

    RETURN QUERY
    SELECT
        sp.full_name, sp.roll_number, ee.student_id,
        es.id, es.status, es.started_at, es.expires_at,
        es.submitted_at, es.last_synced_at, es.security_violations,
        ee.id
    FROM exam_enrollments ee
    INNER JOIN student_profiles sp
        ON sp.user_id = ee.student_id AND sp.deleted_at IS NULL
    LEFT JOIN exam_sessions es
        ON es.exam_id = ee.exam_id
        AND es.student_id = ee.student_id
        AND es.status IN ('active', 'submitted', 'expired', 'terminated')
    WHERE ee.exam_id = p_exam_id
    ORDER BY sp.full_name ASC;
END;
$$;

```

## FUNCTION: get\_exam\_report

On-demand post-exam report for admin. Computes class summary and per-student breakdown from stored results. Does NOT re-evaluate student\_answers — uses pre-computed exam\_results only.

```

CREATE OR REPLACE FUNCTION get_exam_report(p_exam_id uuid)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
STABLE
SET search_path = public, pg_catalog
AS $$
DECLARE
    v_summary jsonb;
    v_students jsonb;
    v_question_analysis jsonb;
BEGIN
    IF NOT is_admin() THEN
        RAISE EXCEPTION 'UNAUTHORIZED';
    END IF;

    -- Class-level summary
    SELECT jsonb_build_object(
        'exam_id', p_exam_id,
        'enrolled_count', COUNT(DISTINCT ee.student_id),
        'submitted_count', COUNT(DISTINCT er.student_id),
        'avg_percentage', ROUND(AVG(er.percentage), 2),
        'highest_percentage', MAX(er.percentage),
        'pass_count', COUNT(*) FILTER (WHERE er.is_passed = true),
        'fail_count', COUNT(*) FILTER (WHERE er.is_passed = false)
    ) INTO v_summary
    FROM exam_enrollments ee

```

```

LEFT JOIN exam_results er
  ON er.exam_id = ee.exam_id AND er.student_id = ee.student_id
WHERE ee.exam_id = p_exam_id;

-- Per-student results with rank
SELECT jsonb_agg(
  jsonb_build_object(
    'rank', ROW_NUMBER() OVER (ORDER BY er.percentage DESC),
    'student_id', sp.user_id,
    'full_name', sp.full_name,
    'roll_number', sp.roll_number,
    'total_score', er.total_score,
    'max_score', er.max_score,
    'percentage', er.percentage,
    'is_passed', er.is_passed,
    'time_taken_seconds', er.time_taken_seconds,
    'session_id', er.session_id
  ) ORDER BY er.percentage DESC
) INTO v_students
FROM exam_results er
INNER JOIN student_profiles sp ON sp.user_id = er.student_id
WHERE er.exam_id = p_exam_id;

RETURN jsonb_build_object(
  'summary', v_summary,
  'students', COALESCE(v_students, '[]'::jsonb)
);
END;
$$;

```

#### ■ INFO

AUTHORIZATION NOTE: `get_leaderboard()` is intentionally unrestricted at the database layer. The application layer handles visibility via the `show_leaderboard_after` exam setting. Explicitly documented to acknowledge this intentional design.

## FUNCTION: `get_leaderboard`

Returns top 50 for a given exam. Called from result screen and admin reports. Respects the `show_leaderboard_after` setting in application layer.

```

CREATE OR REPLACE FUNCTION get_leaderboard(p_exam_id uuid)
RETURNS TABLE (
  rank          bigint,
  student_id    uuid,
  full_name     text,
  total_score   numeric,
  percentage    numeric,
  submitted_at  timestamptz
)
LANGUAGE plpgsql
SECURITY DEFINER
STABLE
SET search_path = public, pg_catalog
AS $$
BEGIN
  -- Validate leaderboard publication state
  IF NOT EXISTS (
    SELECT 1 FROM exams
    WHERE id = p_exam_id
    AND status IN ('active', 'completed')
    AND (
      settings->>'show_leaderboard_after' = 'always' OR
      (settings->>'show_leaderboard_after' = 'exam_end' AND (status = 'completed' OR type = 'practice'))
    )
  ) THEN
    RAISE EXCEPTION 'LEADERBOARD_UNAVAILABLE';
  END IF;

```



```

RETURN QUERY
SELECT
  ROW_NUMBER() OVER (ORDER BY er.percentage DESC, er.computed_at ASC),
  er.student_id,
  sp.full_name,
  er.total_score,
  er.percentage,
  es.submitted_at
FROM exam_results er
INNER JOIN student_profiles sp
  ON sp.user_id = er.student_id AND sp.deleted_at IS NULL
INNER JOIN exam_sessions es ON es.id = er.session_id
WHERE er.exam_id = p_exam_id
ORDER BY er.percentage DESC, er.computed_at ASC
LIMIT 50;
END;
$$;

```

## FUNCTION: admin\_force\_submit\_session

Force submits a session. Requires admin privileges.

```

CREATE OR REPLACE FUNCTION admin_force_submit_session(
  p_session_id uuid,
  p_admin_id   uuid
)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_catalog
AS $$
DECLARE
  v_session exam_sessions%ROWTYPE;
BEGIN
  -- Authorization
  IF NOT is_admin() THEN
    RAISE EXCEPTION 'UNAUTHORIZED';
  END IF;

  -- Fetch and lock session
  SELECT * INTO v_session
  FROM exam_sessions
  WHERE id = p_session_id
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'NOT_FOUND';
  END IF;

  -- Only force-submit active or expired sessions
  IF v_session.status NOT IN ('active', 'expired') THEN
    RAISE EXCEPTION 'SESSION_INVALID: status=%', v_session.status;
  END IF;

  -- Compute and store result (idempotent)
  PERFORM compute_and_store_result(p_session_id, v_session.student_id);

  -- Mark submitted
  UPDATE exam_sessions
  SET status = 'submitted', submitted_at = now()
  WHERE id = p_session_id;

  -- Audit log
  INSERT INTO audit_logs (actor_id, actor_role, action, resource_type, resource_id, metadata)
  VALUES (
    p_admin_id, 'admin', 'admin.force_submitted',
    'exam_session', p_session_id,
    jsonb_build_object('student_id', v_session.student_id, 'exam_id', v_session.exam_id)
  );

  RETURN jsonb_build_object('session_id', p_session_id, 'status', 'submitted');
END;

```

```
$$$;
```

## FUNCTION: `expire_stale_sessions`

Schedule this as a cron job every 5 minutes via Supabase Dashboard > Database > Cron Jobs. Marks expired active sessions and logs the event.

```
CREATE OR REPLACE FUNCTION expire_stale_sessions()
RETURNS integer
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE
    v_count integer;
BEGIN
    WITH updated_rows AS (
        UPDATE exam_sessions
        SET status = 'expired', updated_at = now()
        WHERE status = 'active'
        AND expires_at < now() - interval '2 minutes'
        RETURNING id, student_id
    )
    INSERT INTO audit_logs (actor_id, actor_role, action, resource_type, resource_id)
    SELECT student_id, 'system', 'exam.session_expired', 'exam_session', id
    FROM updated_rows;

    GET DIAGNOSTICS v_count = ROW_COUNT;
    RETURN v_count;
END;
$$$;
```

```
-- Schedule via Supabase Dashboard > Database > Cron Jobs:
-- Name: expire_stale_sessions
-- Schedule: */5 * * * * (every 5 minutes)
-- Command: SELECT expire_stale_sessions();
```

## SECTION 13 FUNCTION GRANTS & RPC ACCESS

### ■ INFO

These GRANT statements allow authenticated Supabase users to call functions via RPC. Functions are SECURITY DEFINER so they run as the DB owner — the security is inside each function body, not at the RLS layer.

```
-- Allow authenticated users to call all public functions via Supabase RPC
GRANT EXECUTE ON FUNCTION create_exam_session TO authenticated;
GRANT EXECUTE ON FUNCTION submit_exam_session TO authenticated;
GRANT EXECUTE ON FUNCTION admin_force_submit_session TO authenticated;
GRANT EXECUTE ON FUNCTION upsert_question TO authenticated;
GRANT EXECUTE ON FUNCTION publish_exam TO authenticated;
GRANT EXECUTE ON FUNCTION get_monitoring_data TO authenticated;
GRANT EXECUTE ON FUNCTION get_exam_report TO authenticated;
GRANT EXECUTE ON FUNCTION get_leaderboard TO authenticated;
GRANT EXECUTE ON FUNCTION update_force_password_change TO authenticated;
REVOKE EXECUTE ON FUNCTION is_admin FROM authenticated;
```

## SECTION 14 STORAGE BUCKET SETUP

### ■ INFO

Create the student-photos bucket in Supabase Dashboard > Storage > New Bucket. Then apply these RLS policies via SQL Editor.

### Create bucket (Dashboard steps)

1. Go to Supabase Dashboard → Storage → New Bucket
2. Name: student-photos
3. Public: OFF (private)
4. File size limit: 2 MB
5. Allowed MIME types: image/jpeg, image/png, image/webp

### Storage RLS policies

```
-- Students access only their own photos (folder = their user_id)
CREATE POLICY "student_photos_read_own" ON storage.objects
  FOR SELECT TO authenticated
  USING (
    bucket_id = 'student-photos'
    AND auth.uid()::text = (storage.foldername(name))[1]
  );

CREATE POLICY "student_photos_upload_own" ON storage.objects
  FOR INSERT TO authenticated
  WITH CHECK (
    bucket_id = 'student-photos'
    AND auth.uid()::text = (storage.foldername(name))[1]
  );

-- Admins have full access to student photos
CREATE POLICY "admin_storage_all" ON storage.objects
  FOR ALL TO authenticated
  USING (bucket_id = 'student-photos' AND is_admin())
  WITH CHECK (bucket_id = 'student-photos' AND is_admin());
```

## SECTION 15 VERIFICATION QUERIES

### ■ INFO

Run these after setup to confirm everything is configured correctly. All checks should return the expected values.

### Verify all tables exist with RLS enabled

```
-- Expected: 15 rows, all with rowsecurity = true
SELECT tablename, rowsecurity
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY tablename;
```

### Verify all indexes were created

```
-- Check index count per table
SELECT tablename, COUNT(*) AS index_count
FROM pg_indexes
WHERE schemaname = 'public'
GROUP BY tablename
ORDER BY tablename;
```

### Verify all functions exist

```
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'public'
  AND routine_type = 'FUNCTION'
ORDER BY routine_name;
-- Expected: create_exam_session, expire_stale_sessions, get_exam_report,
--           get_leaderboard, get_monitoring_data, is_admin,
--           publish_exam, set_updated_at, submit_exam_session, upsert_question
```

### Security verification — run as a test student user

```
-- If running as a student (anon/authenticated with student role):
-- These should ALL return 0 rows (RLS blocking access):
SELECT COUNT(*) AS question_banks_visible FROM question_banks;
SELECT COUNT(*) AS questions_visible      FROM questions;
SELECT COUNT(*) AS question_options_visible FROM question_options;
SELECT COUNT(*) AS other_answers           FROM student_answers
  WHERE session_id NOT IN (
    SELECT id FROM exam_sessions WHERE student_id = auth.uid()
  );
-- These should return > 0 for your own data:
SELECT COUNT(*) AS my_profile FROM student_profiles WHERE user_id = auth.uid();
```

This SQL reference covers the complete production database for the AVIORA Examination Portal. For a new tenant, run every section in order in a fresh Supabase project. The schema is deliberately generic — no aviation-specific column names — making it reusable across clients.

AVIORA Examination Portal · ZYXEN · 23 July 2026 · v1.0